

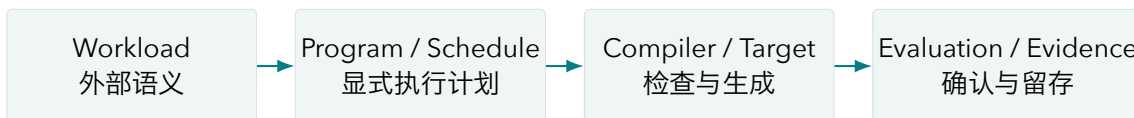
研究报告 / 技术体系与实验证据

# 面向 GPU 内核优化的 智能体与编译器协同演进

Agent-Compiler Co-Evolution for GPU Kernel Optimization

秦海岩 Haiyan Qin  
北京航空航天大学

2026 年 9 月 23 日  
阅读快照 / 源码 ff02b8fe



**报告边界** 正文所读源码固定为 ff02b8feadf4c7d1399f818c5283830280d7cb85。历史设备实验仍绑定各自的源码、精确目标、Workload、oracle 和计时协议；文内引用指向所读的固定提交。

## 摘要

**问题。**智能体能提出 GPU 内核的候选实现，但一次失败或一项延迟读数，难以说明问题属于调度、表达能力、目标准入还是测量质量。

**方法。**系统用带类型的执行计划描述单个内核，以完整程序组织阶段。编译器定位错误；实验系统固定任务和预算；评测检查外部答案、设备计时与性能剖析；证据系统留存原始观察。重复缺口在运行之外由后继提交修复。

**证据。**独立实现已声明五家厂商的八个精确目标，当前源码的 179 项语料检查符合预期。NVIDIA、Apple、Hygon 和 MetaX 各有范围不同的设备结果；AMD 的计时边界仍待解决。与 CAKE 导出产物重叠的 TinyGEMM2 固定形状已通过严格数值门，但性能差距显著。跨硬件知识处理的增量收益尚无实机因果估计。

**关键词：**GPU 内核；中间表示；静态验证；智能体；实验可复查性；跨硬件优化

## ABSTRACT

open-cake-ir independently implements a CAKE-inspired system in which agents author typed, hardware-explicit schedules, receive localized compiler feedback, and validate candidates against external oracles and target-specific measurements.

A corpus-gated compiler successor is introduced between frozen optimization runs. The implementation declares eight exact targets across five vendors; device correctness and measurement coverage remain uneven. A same-shape TinyGEMM2 comparison passes strict numerical checks while exposing a substantial performance gap. Cross-hardware knowledge-transfer effects remain unmeasured. The software snapshot is commit `ff02b8fe`; each historical measurement retains its own source and assay boundary.

## 阅读约定

**软件合同。**Corpus Gate 与离线 lowering 说明源码在声明范围内符合预期。

**设备观察。**正确性、计时质量、局部收益与完整 Program 分别绑定自己的目标、Workload、运行提交和收据。

**研究结论。**不同作者环境或知识处理的效果，只有预先分配的独立 Run 和共同确认协议能够估计。历史快照与论文报告值始终以来源标签区分。

## 内容导览

|                                                 |           |                                             |           |
|-------------------------------------------------|-----------|---------------------------------------------|-----------|
| <b>1 问题、研究对象与论点</b>                             | <b>3</b>  | 6.2 双时间尺度与 Finding 的负责位置 . . .              | 10        |
| 1.1 背景：公式、执行计划与智能体反馈                            | 3         | 6.3 研究对照的单位和参考访问 . . . . .                  | 11        |
| 1.2 CAKE 论文提供的研究起点 . . . . .                    | 3         | 6.4 证据保存、回放与结论资格 . . . . .                  | 11        |
| 1.3 本工作的定位与当前贡献 . . . . .                       | 4         | 6.5 结论阶梯 . . . . .                          | 11        |
| <b>2 设计原则与实现边界</b>                              | <b>4</b>  | <b>7 实现覆盖与代表性设备案例</b>                       | <b>11</b> |
| <b>3 系统合同与职责边界</b>                              | <b>4</b>  | 7.1 平台覆盖是分层事实 . . . . .                     | 11        |
| 3.1 从任务到可审计结论 . . . . .                         | 4         | 7.2 NVIDIA：任务族覆盖与外部比较 . .                   | 12        |
| 3.2 Compiler：独立解释完整程序 . . . .                   | 5         | 7.3 Apple、Hygon、AMD：不同结论等级                  | 12        |
| 3.3 Lab 与研究分配：冻结作者能看什<br>么、能花多少 . . . . .       | 5         | 7.4 MetaX：单内核、完整输出和实际<br>Run 分别验收 . . . . . | 12        |
| 3.4 Evaluation 与 Evidence：判断和留存<br>分开 . . . . . | 6         | 7.5 案例 A：C550 的局部 tiling 收益 . . .           | 13        |
| 3.5 两个时钟：候选搜索与编译器演进 .                           | 6         | 7.6 案例 B：B300 的对齐条件与质量门 .                   | 13        |
| <b>4 程序表示：任务、完整程序与执行计划</b>                      | <b>6</b>  | 7.7 两个局部案例的结论范围 . . . . .                   | 13        |
| 4.1 显式的计划，而非新的布局代数 . . .                        | 6         | 7.8 历史 B200 案例的放置方式 . . . . .               | 14        |
| 4.2 受限 Python 前端与同一规范 IR . . .                  | 7         | <b>8 与 CAKE 的对照：方法、重叠任务与差距</b>              | <b>14</b> |
| 4.3 一个数值反例：FMA 与乘加 . . . . .                    | 7         | 8.1 TinyGEMM2：同形状的诚实差距 . .                  | 14        |
| 4.4 完整 Program 的阶段与绑定 . . . . .                 | 7         | <b>9 跨硬件知识迁移：机制与待验证估计量</b>                  | <b>15</b> |
| 4.5 精确目标文档 . . . . .                            | 8         | 9.1 可复用的单位与负责位置 . . . . .                   | 15        |
| 4.6 表示变更的验收义务 . . . . .                         | 8         | 9.2 E/P 四组实验 . . . . .                      | 15        |
| <b>5 编译器：从局部化检查到目标源码</b>                        | <b>8</b>  | 9.3 已到达工具链的离线迁移实例 . . . .                   | 16        |
| 5.1 分层拒绝与反馈 . . . . .                           | 8         | <b>10 限制、可复查性与结论</b>                        | <b>16</b> |
| 5.2 源码生成与下游工具链 . . . . .                        | 8         | 10.1 解释结果时的四项限制 . . . . .                   | 17        |
| 5.3 Target 与生成路线如何保持分离 . . .                    | 8         | 10.2 下一步的可检验目标 . . . . .                    | 17        |
| 5.4 从 Finding 到物理资源观察 . . . . .                 | 9         | <b>A 术语、源码入口与完整示例</b>                       | <b>18</b> |
| 5.5 Corpus Gate 的含义 . . . . .                   | 9         | <b>B 结果来源与历史身份</b>                          | <b>19</b> |
| <b>6 实验方法与证据等级</b>                              | <b>10</b> | <b>C 复现边界与报告构建</b>                          | <b>19</b> |
| 6.1 一次 Run 的四阶段 . . . . .                       | 10        |                                             |           |

## 1 问题、研究对象与论点

本文的智能体 (Agent) 负责提出和修订执行计划; GPU 内核是在加速器上并行执行的计算程序; Compiler 检查计划并生成目标代码。给定同一数学算子, GPU 实现仍须决定任务分块、线程分工、存储层次、访问坐标、同步与执行顺序。一个只输出延迟的搜索过程很难说明某次失败属于候选调度、编译器表达能力、硬件准入, 还是测量噪声。CAKE 论文提出让 Agent 编辑显式执行计划, 并以编译器分析和外部评测形成反馈闭环 [1]。open-cake-ir 是对这一研究路线的独立工程实现; 论文报告的模型实验和速度数字不作为本仓库结果 [6]。

本报告的研究对象是**可复查的 Agent 编写环境**: 一份候选包含什么决策, 系统能在什么范围内拒绝, 设备证据如何决定接受, 以及真实缺口怎样改变下一版 Compiler。核心问题可以表述为:

**核心问题** 在保持 Workload 语义和实验条件可追溯的前提下, 怎样让 Agent 的 Kernel 决策可检查、让失败有负责位置, 并让重复经验进入后继编译能力?

本文主要陈述三项已实现的机制: **显式的 Program/Schedule 合同、分层诊断与目标精确准入、冻结运行和追加式证据**。跨硬件知识迁移的增量价值是后续实验问题, 不是现有软件实现的推论。

### 1.1 背景: 公式、执行计划与智能体反馈

同一按行归约公式可以采用不同的行块、执行组、存储位置和归约顺序。候选从数学式走向设备内核时, 正确性义务没有消失, 反而增加了地址域、同步、资源和目标指令等约束。若编写环境只给出编译错误、数值通过位和一个延迟, 智能体难以区分三种不同的下一步: 修复当前候选、改进生成路线, 或补足表示与检查规则。资源分析与 profiler 可以缩小解释空间, 但不能自动证明唯一原因。本文因此沿着任务语义、执行决策、检查反馈、设备观察四层组织叙事。

编写 GPU 内核还需要区分两种结果。一个固定形状上更快的内核属于该任务的优化; 将同一机制用于另一形状或硬件, 要重新验证目标资源、数值和计时; 将其提升为通用编译器规则, 还须说明适用条件、拒绝反例和后继任务中的作用。把这三个层次混在一张加速比表中, 会掩盖系统能力演进本身需要的证据。

### 1.2 CAKE 论文提供的研究起点

CAKE 论文将智能体编写的硬件执行计划、编译前分析与外部评测放在同一个循环中。其 Cake IR 显式记录资源、角色、访问和同步, 而把屏障地址、阶段位等机械细节交给 lowering; 布局没有成为作者必须操作的一阶代数。编译器以程序安全、硬件符合性、数据一致性和调度语义的局部诊断筛选候选, 成本模型只在有目标校准时用于设备时间之前的排序。反复出现的真实失败推动后继 IR、Verifier、lowering 或校准变化; 最终正确性和速度仍由外部参考与设备测量决定 [1]。

该论文的代表性因果实验是在 B200 的固定 Flash-KMeans 形状上, 以隐藏目标低层实现的起点比较 Cake IR 编写环境和直接 CUDA/PTX 编写环境。论文报告每组三次独立运行、共同的 80M provider-token 截点, 并把结果归于整套编写环境, 而非孤立的 IR 语法。其性能、KDA 服务与 dispatcher 结果属于原论文的报告值; 本仓库不能把它们作为本地复测数据。CAKE 的公开论文还说明目标主要是 NVIDIA Ampere 至 Blackwell, 非 NVIDIA 后端的实现与迁移成本没有在该论文中测得 [1, 6]。

### 1.3 本工作的定位与当前贡献

open-cake-ir 是沿这一问题重新构造的独立系统。它把 Compiler 做成可单独调用的核心，使 Workload、研究处理、作者控制和证据监管分别由明确的层负责；Target 以精确文档声明硬件事实，五个 Vendor 在同一公共表示下各自拥有生成和执行边界。这样的实现提供了检查不同目标与不同来源证据的公共接口，但**公共接口并不意味着同等设备资格或同等性能**。

本文可支持的当前贡献有三项：一是可编写、可检查、可生成的 Program/Schedule 与精确 Target 合同；二是冻结 Run 内优化与 Run 外编译器演进的分离，以及可回放的候选、诊断、确认和终态；三是多目标接入和有界设备案例，为后续跨硬件机制检验提供可运行的研究基础。第三项中的跨硬件增量收益仍是待验证问题，不能由路线接通或一个目标的本机优化推出。第7节将每个结果限定到相应的任务、目标、计时和证据级别。

## 2 设计原则与实现边界

CAKE 在附录 B.1 给出八项 IR 设计原则 [1]。表1把它们转为本仓库编写和修改执行计划时的检查问题，而不是把通过现有用例写成这些原则在所有目标上的证明。

**表 1** P1-P8 对本仓库的实际约束。具体语法和目标准入仍以源码合同为准。

| 原则      | 实现时需要回答的问题                                     |
|---------|------------------------------------------------|
| P1 易于编写 | Python 候选能否用熟悉的变量、表达式和角色作用域表达任务，而不要求作者手算网格元数据？ |
| P2 性能透明 | 分块、执行组、存储、访问与同步等影响性能的决定能否在 Schedule 中读到？       |
| P3 规范形式 | 同一操作是否落到一套规范的类型与操作模型，避免 Python/JSON 两套语义？      |
| P4 静态类型 | 类型、操作数、形状和资源关系能否在构造或 Assessment 时明确拒绝？         |
| P5 可分析  | Verifier 和资源分析所需的关系是否在 IR 中显式存在？               |
| P6 语料门  | 改动是否在接受与拒绝用例上通过完整 Corpus Gate？                 |
| P7 分析一致 | 数据模型变化是否同时更新合法性、资源和性能相关分析？                     |
| P8 硬件根据 | 原语承诺的目标指令、存储和同步行为是否有 Target 与后端根据？             |

本项目另有三条实现边界。**第一，目标精确匹配**。目标未声明的事实报告为未建模，目标不支持的指令或代码对象由其拥有者明确拒绝，不能通过名称相近的架构静默降级。**第二，结构合法与设备正确分开**。静态分析可能有漏报或误报，后端会引入未在逻辑 Schedule 中显式分配的物理资源，实际正确性与延迟须由设备观察决定。**第三，Compiler 与实验处理分离**。编译器不读取 Study 分组、provider 或证据存储；一次冻结 Run 的候选只在同一 Compiler 和 Workload 合同内比较，编译器变化在后继提交及语料验证后进入下一轮 [2, 5]。

## 3 系统合同与职责边界

### 3.1 从任务到可审计结论

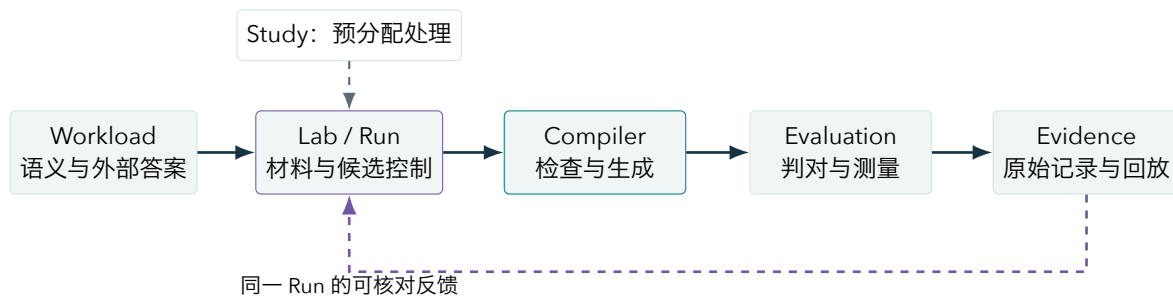
任务合同（Workload Contract）固定数学语义、输入域、外部标准答案和容差。执行计划（Schedule）描述一个内核的资源、执行组、操作、地址与同步；完整程序（Program）再声明公共张量、阶段顺

序和绑定。运行规格（RunSpecification）固定一次优化的源码、目标、材料、工具权限和预算；研究计划（StudyPlan）仅在研究对照时进一步固定处理、分配、终点与分析。这些对象的职责不同，不能用一个候选的局部通过来代替 Study 结论[2, 15]。

**表 2** 一次候选从表达走向结论时的负责位置。某层的通过不自动推出下一层的通过。

| 对象或模块      | 拥有的事实                    | 对外给出的产物                 |
|------------|--------------------------|-------------------------|
| Workload   | 数学、形状、输入分布、外部答案          | 不随候选改变的验收合同             |
| Compiler   | IR 类型、Target、合法性、后端准入与生成 | Assessment、Finding、目标源码 |
| Lab        | 材料、候选搜索、预算与 Run 控制       | 已提交候选和过程事件              |
| Evaluation | 外部正确性、目标计时质量和剖析          | 独立的检查、计时和 profiler 收据   |
| Evidence   | 原始文件、事件、回放与审计            | 可由原始材料重建的报告             |
| Study      | 处理分配、估计量和缺失策略            | 限定研究问题的分析结果             |

Compiler 是独立产品核心，不导入 Lab、provider、Workload 实现或证据库。Lab 调用 Compiler 与 Evaluation，Evidence 保存其观察。这样的依赖关系使同一个 Compiler 可用于离线检查，也防止实验策略偷偷进入 Schedule 的语义判断。系统源码与边界见[架构章节](#)和[职责地图](#)。



**图 1** 任务与研究处理先于候选固定；Lab 调用独立 Compiler、Evaluation 与 Evidence，回放后的反馈才进入同一 Run 的下一候选。箭头表示调用或信息流，不表示前一关通过即可跳过后一关。

### 3.2 Compiler: 独立解释完整程序

Compiler 读取一份完整 Program 及其叶子 Schedule，在同一精确 Target 与同一源码提交上解释类型、绑定、依赖和硬件承诺。Assessment.accepted 表示结构与通用规则的结论，lowering\_eligible 则记录后端能否兑现；两者分开允许可表达但未实现的路线给出明确资格拒绝。显式改写返回完整后继候选或局部拒绝，不让 Lab 持有第二套图语义。只有通过门的计划才由后端确定性生成可检查源码。Compiler 不知道研究组别、provider、设备分配、oracle 或最终 Claim[2, 3]。

### 3.3 Lab 与研究分配：冻结作者能看什么、能花多少

独立工程 Run 的输入是一份 RunSpecification；若研究需要比较条件，StudyPlan 事先分配相同 Run 引擎的处理组、估计量与缺失规则。Lab 在触发作者、工具链和 GPU 之前核对 Workload、Compiler/Executor、provider、基线和预算。作者提交结构化候选，Ralph 控制搜索、编译和确认额度；搜索停止后固定提名，再用新鲜 Evaluation 收据确认，不因确认失败临时换候选。历史 CampaignLock 只在输入边界转换到共同 Run，而不构成另一套执行循环。这样，一次实验的处理从准备到终态均有单一负责位置。

### 3.4 Evaluation 与 Evidence: 判断和留存分开

Evaluation 接受封存的可运行候选，按 Workload 材料化输入，以外部 oracle 判对；正确性通过之后，才按精确目标的计时、状态重置与 profiler 协议收集样本。搜索、独立确认和 profiler 各有自己的收据；CV 质量失败属于测量观察，不把数值正确候选改记为算错。Evidence 保存候选字节、事件顺序与终态，离线审计在不调用 provider、GPU 或网络的条件下重建协议、资格和缺失。Study 使用通过审计的 Run，而不是用作者自述或某个搜索最低读数生成估计。完整 Program 的每个阶段保留编译和 launch 身份，最终输出及程序区间单独验收；单次 dispatch 的计时器不自动具备多阶段 Program 资格[5]。

### 3.5 两个时钟：候选搜索与编译器演进

内循环在冻结的 Compiler、Workload、Target、oracle 和测量合同下进行：生成结构不同的候选，先通过构造检查、Verifier 和后端预检筛选，再对幸存者做设备正确性、计时与剖析，最后把诊断送回下一次候选。外循环由累积的 Finding 触发：若问题属于通用合法性、缺失原语、lowering 或目标校准，维护者在后继提交中修改其负责模块，验证后才开启新 Campaign。一次运行不能边测边改变尺子[2, 5]。

## 4 程序表示：任务、完整程序与执行计划

### 4.1 显式的计划，而非新的布局代数

受限 Python 前端与 JSON 文档形式进入同一规范的 typed Schedule；Python 入口解析受支持的语法，并不执行任意宿主 Python 程序。Schedule 记录执行组、缓冲区、访问映射、循环、操作与同步，性能相关的硬件选择因此可被检查。它没有把 layout 建成一阶代数；对目标硬件的具体存储与访问承诺由相应 Target 和后端解释并验证。Triton 路线仍把部分物理 placement、寄存器分配与指令选择交给下游工具链，不能把其生成结果当作 Schedule 已经精确指定的事实[3, 2]。

例如对一行 Softmax，Workload 给出  $y_j = \exp(x_j - m) / \sum_k \exp(x_k - m)$ ，其中  $m = \max_k x_k$ ；候选 Schedule 另选每个执行组处理的行、列块、归约方式与写回坐标。若读取、归约和写回对同一行的坐标约定不一致，问题属于数据一致性；若两个执行组写同一输出，问题属于程序安全；若目标不提供所需指令，则属于硬件符合性或后端资格。数学式相同不能消除这些执行计划差异。

Program 负责多个 Schedule 的公共输入输出、阶段顺序与张量绑定。完整 Program 的正确性要检查所有阶段的最终输出；单 Kernel 的计时或剖析不能自然推广为 Program、模型或服务收益。

输入：512 行 × 128 列

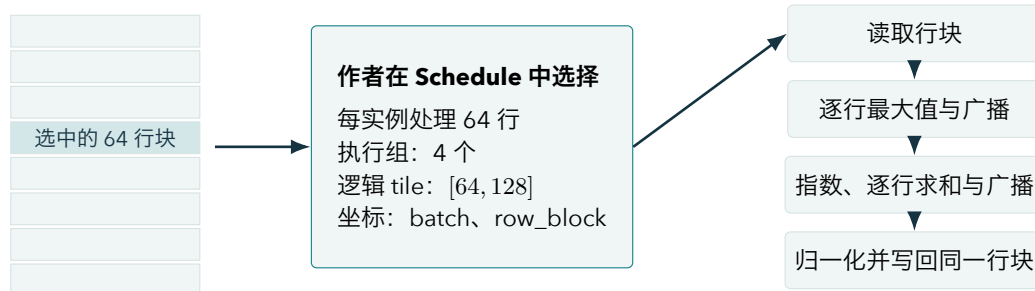


图 2 Softmax 的数学结果由 Workload 固定；行块、执行组、访问坐标与操作顺序是候选 Schedule 中可修改、可检查的决定。改绘自 仓库原图；本图不表示设备计时。

## 4.2 受限 Python 前端与同一规范 IR

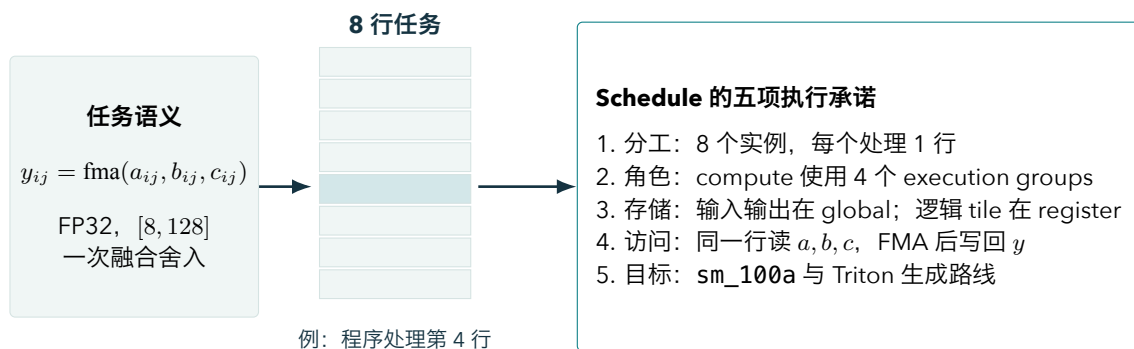
Python 前端允许作者用变量、表达式、角色作用域和分块循环构造 Schedule，同时保留语法树中的源码位置以指回诊断。它只接收受支持的语法并解析 AST，接收候选的入口不执行任意宿主 Python 导入或函数体；JSON 仍是同一规范 Schedule 的序列化形式。正常导入一个受信任的示例与把不受信任的候选交给 AST 入口，是不同的安全边界 [3]。如下摘录自 [当前 Softmax 示例](#)：

```
compute = lm.role(execution_groups=[0, 1, 2, 3])
row_block = lm.program(x, axis=0, dimension=1, tile=64)
batch = lm.program(x, axis=1, dimension=0, tile=1)
with compute:
    x_tile = lm.load(x[batch, row_block, :], reuse="reused", id="load_x")
    rowmax = lm.reduce(x_tile, op="max", axis=1, scope="cta", id="row_max")
    shifted = lm.sub(x_tile, lm.broadcast(rowmax, axis=0), id="shift")
    weights = lm.exp(shifted, id="exponentiate")
    rowsum = lm.reduce(weights, op="sum", axis=1, scope="cta", id="row_sum")
    y_tile = lm.div(weights, lm.broadcast(rowsum, axis=0), id="normalize")
    lm.store(y[batch, row_block, :], y_tile, id="store_y")
```

原示例的完整签名固定输入输出为 [8, 512, 128] FP32，并指定 **sm\_100a** 与 Triton 生成路线；这里仅省略装饰器和函数签名以突出工作划分与数据流。**tile=64** 是每实例处理的行数，四个执行组是调度承诺；它们并不从 Softmax 公式自动得出。行最大值和行和必须显式广播到列维后才能与输入 tile 对齐，写回仍使用相同行坐标。调整这些选择后，Compiler 重新检查，设备评测再决定正确性和收益。

## 4.3 一个数值反例：FMA 与乘加

FMA 的数学关系可写成  $y_{ij} = a_{ij}b_{ij} + c_{ij}$ ，但一次融合舍入和先乘后加的两次舍入不是同一数值合同。当前示例对 [8, 128] FP32 输入显式请求 **ptx.fma.rn.f32**；Target 和后端拥有该指令的准入与拼写，外部 oracle 拥有最终数值验收。图3复用仓库已有的工作示意；高亮行只用于说明一个程序实例的访问坐标，不是硬件寄存器数量或设备执行截图 [3]。



**图 3** 同一 FMA 公式没有规定工作分配、存储或生成路线。图按 [仓库原图](#) 重新排成适合报告的矢量图；逻辑 register tile 的长度不等于每线程物理寄存器数，也不表示新的设备测量。

## 4.4 完整 Program 的阶段与绑定

一个 Program 声明公共张量、各阶段顺序以及输入输出绑定，叶子阶段包含完整 Schedule。例如选择性注意力程序需要池化、评分、Top- $k$ 、索引展开和注意力阶段，并在循环中保存选择与归约状态；单 Kernel 合格不能替代阶段间张量一致性与最终输出的检查。程序级变换可以改变阶段数量，所以比较前须固定 Workload 与终点，再以完整 Program 的调用路径判对与计时。第7节把历史 QSA

程序作为这一范围的案例，而不把它当成通用 Program 自动优化已经完成的证据。

## 4.5 精确目标文档

当前源码声明八个精确目标，分属 NVIDIA、Apple、AMD、Hygon 和 MetaX 五个 Vendor。精确目标文档 (Target) 拥有自己的线程组宽度、存储空间、指令合同、代码对象及资源事实；未声明的事实报告为“未建模”，不借用相似目标的常数。后端是生成机制，不是厂商身份：现有 Triton、Metal、原生 CUDA 与 CuTe DSL 四条路线，各自声明可生成的代码对象。编译器对目标和后端做精确匹配，不能把未声明目标降级到邻近架构 [4, 2]。

## 4.6 表示变更的验收义务

新增 IR 原语须同时给出类型、效应、合法性、静态分析和目标行为；只有语法而没有这些语义，不能成为已准入能力。仓库将易编写、性能决策可见、规范形式、构造时类型检查、可分析、语料门、分析一致与硬件根据作为表示的八项设计约束 [1, 3]。某算子若能由现有原语组合实现，应先添加 Workload Contract；只有真实表达缺口才要求扩大 Compiler。

# 5 编译器：从局部化检查到目标源码

## 5.1 分层拒绝与反馈

Compiler 首先解析与构造类型正确的 IR，然后按四类合同检查：

|                                      |                                         |
|--------------------------------------|-----------------------------------------|
| <code>schedule_semantics</code> 调度语义 | <code>hardware_conformance</code> 硬件符合性 |
| <code>data_consistency</code> 数据一致性  | <code>program_safety</code> 程序安全        |

Assessment 中的 Finding 指向字段或源码区域，带稳定代码、类别、严重度和阻断范围；非阻断的 guidance 与拒绝原因分开。后端预检负责该生成路线特有的能力边界。静态分析只能在已建模的范围内阻断；未声明的资源上限和下游产生的物理资源需求需要另行观察 [2, 5]。

一个可复查的小反例是 FMA 候选漏掉第三个输入：构造仍指向同一数学操作，Assessment 在 `operations[3].reads` 报 `ELEMENTWISE_ARITY`，指出该操作需要三个输入。作者因此可以修复具体数据边，无需先耗费设备时间。通过静态检查只允许候选继续；它不证明工具链编译、GPU 数值正确或速度收益 [2]。

## 5.2 源码生成与下游工具链

合法 Schedule 经选定后端输出可检查的目标源码。Triton 路线再通过厂商对应的 Triton 工具链编译；Metal、原生 CUDA 与 CuTe DSL 路线各保留自己的预检与生成条件。Target 声明的代码对象随生成路线进入工具链和 Executor，不能由目标名字猜测架构。生成源码、编译产物、设备可加载性、外部正确性与实测延迟是五个递进但互不等同的检查点 [2]。

## 5.3 Target 与生成路线如何保持分离

八个声明目标属于五个平等的 Vendor；同一 Triton emitter 可以服务 NVIDIA、AMD、Hygon 和 MetaX，因而后端名不是厂商名。表3把目标文档的代码对象与当前源码生成机制放在一起；它描述可尝试

的路线，不把路线存在写成每个算子都已获得设备正确性或计时资格。精确的支持与拒绝由目标文档、后端预检和对应工具链共同决定。

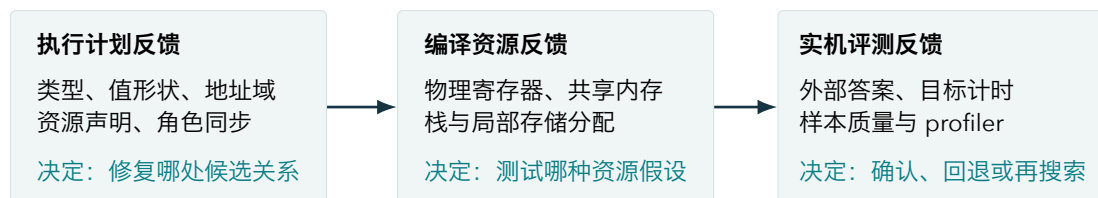
**表 3** 固定源码中的目标与生成路线。代码对象来自各自 Target 文档；设备证据另见第7节。

| Vendor | 精确目标 ID                                                       | 代码对象          | 当前主要源码路线                  |
|--------|---------------------------------------------------------------|---------------|---------------------------|
| NVIDIA | sm_100a, sm_103a                                              | CUBIN         | Triton; 原生 CUDA; CuTe DSL |
| Apple  | apple_gpu_family7,<br>apple_gpu_family8,<br>apple_gpu_family9 | Metal archive | Metal                     |
| AMD    | gfx1151                                                       | HSACO         | Triton / ROCm             |
| Hygon  | gfx938                                                        | HSACO         | Triton / DTK-HCU          |
| MetaX  | xcore1002                                                     | MCFATBIN      | Triton / MACA             |

目标文档一次声明 warp/wave 宽度、资源和指令事实；后端只保留自身的生成拼写与资格。一个 Target 未声明寄存器上限等事实时，分析返回明确的未建模报告，不默认成相邻 GPU 的值。MetaX 路线的 `triton_arch=80` 是工具链兼容 API 值，并不是 NVIDIA SM80 身份。一个新的同厂目标可由新文档加入，但目标集合、指令合同和后端证据门仍需有意识地审查与更新 [4, 2]。

## 5.4 从 Finding 到物理资源观察

Assessment 把阻断性 Finding、仅报告的覆盖限制及非阻断 guidance 分开。它能告诉作者哪个字段违反了哪类合同，也能在缺少目标校准时拒绝给出伪精确的成本顺序。逻辑 Schedule 中的寄存器 tile 不等于编译后每线程寄存器数；后端可能引入暂存、栈或共享内存。工具链产物的资源报告可以在不启动 GPU 的条件下收集，实机 profiler 才能观察目标运行时瓶颈。两类观察以候选和源码身份关联，但不互相替代。详见 [编译资源分析与校准边界](#)。



**图 4** 三类反馈回答不同问题。按 [仓库既有说明图](#) 改绘；箭头表示信息逐层增加，不能由资源估计直接推出设备速度，也不表示失败会自动生成唯一的 Compiler 修改。

下面的离线入口使用当前项目的 Compiler 配置读取 FMA Python 示例，只检查并生成源码；它不编译二进制、不启动设备，也不会产生性能样本：

```
PYTHONPATH=src python3 -m open_cake_ir.cli compiler assess \
  examples/python/fma.py --format text
PYTHONPATH=src python3 -m open_cake_ir.cli compiler lower \
  examples/python/fma.py --output /tmp/cake-fma.py --format text
```

## 5.5 Corpus Gate 的含义

固定源码 `ff02b8fe` 的状态页报告 **179/179** 项语料检查符合已审查预期；其中 Apple family7 与 family9 是已声明但未被该语料检查的目标。该数值证明当前软件合同与预期相符，不证明这些目

标的设备运行或性能。语料预期必须作为独立审查决定维护，不能为使 Gate 变绿而在验证前重生或 [7, 5]。

## 6 实验方法与证据等级

### 6.1 一次 Run 的四阶段

系统按下列顺序运行候选搜索：

1. 生成结构不同的 Schedule 或 Program 候选，并记录具体变更；
2. 以 IR 构造、Verifier、backend preflight 和有目标覆盖的成本估计筛选，未覆盖时明确弃权；
3. 对幸存者使用外部 oracle 检查完整输出，再在目标计时器、状态重置与质量门下测量，并保留 profiler 观察；
4. 将失败归于候选、Verifier、成本模型、IR 或 lowering，决定继续搜索或登记后继 Finding。

成本估计用于设备时间之前的排序与过滤，不能决定最终接受。每个目标分别声明计时器、测量区间和样本前设备状态；缺少其中一项时应报告测量覆盖限制，而不是继承其他目标的延迟语义。

### 6.2 双时间尺度与 Finding 的负责位置

图5把两类变化隔开。在内循环，一个 Run 固定 Workload、Compiler、目标、作者可见材料和测量合同；作者可修改候选，外部系统追加诊断、正确性和计时收据。在外循环，维护者根据保留的反例判断应改变候选、Verifier、IR、lowering、目标事实或成本模型。若要改 Compiler，改动在新提交及语料验证后才供下一次 Run 使用；原 Run 的解释不随源码后来移动。一次性能差距也可能得出 No promotion，不自动铸造成 pass。

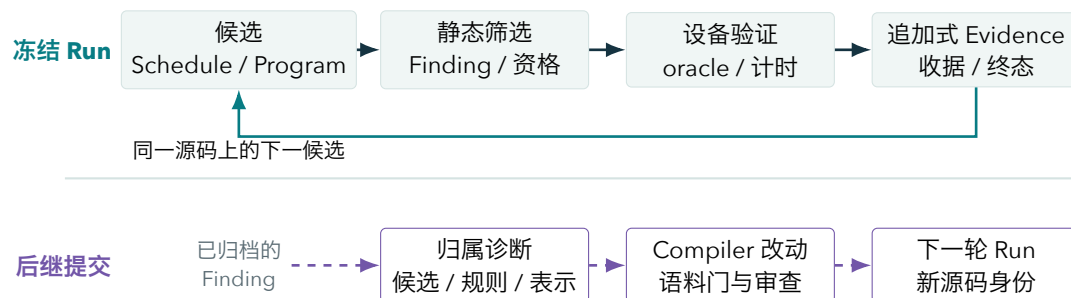


图5 同一 Run 内只改变候选；运行之外的证据驱动下一版 Compiler。下行虚线表示经 Finding 归属、提交和语料门之后的可能改动，不表示每次失败都会产生新原语或收益。

最新集成源码还保留一份 C550 M17 的十轮 Agent Run：冻结源码 896e0887 的 Evidence 记录十次作者回合、九次候选拒绝和终态；第二轮的 ACCESS\_TILE\_MISMATCH 指向具体访问映射，第六轮来源的候选后来通过五类输入的独立正确性检查及 MCPTI 成对确认，Run 内固定基线与候选中位数为 12.032/11.008  $\mu$ s。它验证了一次反馈、筛选、确认与封存的实际路径；运行容器不具备整机独占资格，结果限于 local\_serialized，更不能据此归因于反馈相对其他作者环境的额外效果。记录及其边界由架构案例拥有。

### 6.3 研究对照的单位和参考访问

正式比较的实验单位是独立 Run，不是 Turn、候选或检查点。干净起点只能看到规格、oracle、高层代码和被授权的参考资料，不能读取目标的完整低层实现；已知 Kernel 复现则可以读取该实现。直接低层编写与 Cake 编写是 Authoring Environment 的不同处理条件，需要共同的目标、任务、模型、预算与评测协议。搜索内的最佳读数须经候选固定后的独立确认；基础设施故障应保留为缺失，不能重写成数值错误 [6, 15]。

### 6.4 证据保存、回放与结论资格

候选、评测和终态以追加方式保存，报告由原始事件、产物和收据派生。一次 CPU 或离线回放可以重新计算选择和终点状态，但不能替代当时的设备样本。archive\_integrity 验证归档内容是否能按协议回放，filesystem\_custody\_verified 验证写入和发布环境的监管前提；两者相互独立。若后一条件缺失，即便 Git 检出内容可读，也不能把它提升为资格、估计或性能晋升。历史证据在其原提交和原协议下解释，不由本报告的较新编译器重新赋予资格 [5]。

### 6.5 结论阶梯

表4 是本文解读每个案例的最小证据门。任一层的通过只授予该层的陈述资格。

**表 4** 结论阶梯：“通过”不沿表格自动向下传递。

| 可陈述的结果        | 必要证据                             | 尚需另行验收        |
|---------------|----------------------------------|---------------|
| 结构与目标准入       | typed IR、精确 Target、Finding 与后端预检 | 工具链编译和加载      |
| 设备正确性         | 完整输出对外部 oracle，覆盖规定形状与分布         | 计时质量和性能收益     |
| 局部性能          | 固定候选与基线、共同计时区间、质量通过              | 跨任务或跨硬件泛化     |
| Program 或服务收益 | 完整调用路径、dispatch 与应用层验收           | Agent 处理的因果效果 |
| 研究归因          | 预注册处理、独立 Run、共同确认与缺失规则           | 未测试任务与平台的外推   |

Evidence 保存候选、原始样本和事件；报告是由这些记录派生的视图。源码身份由干净 Git 提交决定，Compiler 为 open-cake-ir@<commit>，Executor 为 <target>@<commit>。本报告的阅读提交只界定当前叙述所见的软件，不能代替历史实验的执行提交或文件监管条件。

## 7 实现覆盖与代表性设备案例

### 7.1 平台覆盖是分层事实

表5 概括仓库已发布的代表性证据。它不是跨厂商性能排行；同一平台的不同 Workload 也可能有不同的计时和确认资格。完整细节由各平台专题与原始收据负责 [2]。

表 5 截至报告快照的代表性平台能力与主要解释边界。

| 平台     | 已报告的执行范围                                        | 需保留的边界                                  |
|--------|-------------------------------------------------|-----------------------------------------|
| NVIDIA | B300 的单 Kernel、部分完整 Program；<br>B200 另有固定案例     | FlashInfer 外部比较有通过与未通过质量门的边，按任务解读       |
| Apple  | Metal 固定算子与 TaskLab 优化记录                        | 已验收设备和计时口径不推广到全部 family 或完整 Program     |
| AMD    | gfx1151 设备调查与 smoke                             | 计时器绝对时间未对齐，尚无合格的完整 Agent 收益结论           |
| Hygon  | BW1101 / gfx938 固定任务与 HIP 路线                    | 短 Kernel 的计时分辨能力限制部分比较                  |
| MetaX  | C550 固定单 Kernel 及若干完整输出；单 Kernel 配对计时和 profiler | 完整 Program 普通 Run 计时与完整 provider 闭环仍需验收 |

## 7.2 NVIDIA：任务族覆盖与外部比较

FlashInfer 逐任务综述把 B300 的 26 个任务按同一固定形状下的原 starter、改写产物和外部参考分别保留。截至该专题的 2026-09-21 快照，16 项已有三方数值比较，其代表产物或保留的原 starter 对外部边中：2 项合格领先、6 项 close\_null、4 项合格落后、4 项没有合格外部边；另 10 项尚未完成外部性能闭环。这是任务状态分布，**不是**把不同形状和不合格边合成的平均加速比。任务 022 的改写候选对外部有合格领先，但相对原 starter 的直接比较未通过 CV 门，不能因此自动取代 starter。任务 003 的整行改写在局部对照上改善，对外部仍合格落后；这些负例使差距可定位。

B300 服务实验、CTA 宽度验证、FlashInfer 改写与 CAKE 任务复现拥有各自的基线、版本与测量协议。把这些行并排作为平台观察可以展示研究覆盖；求一个“B300 总加速比”会改变各行的含义。B200 的 FMA、状态操作和 QSA 记录另按原提交保留，不能把 B300 的设备结果移给 B200。独立 NCU 剖析用于解释资源和工作分配，不修复某一条 CUPTI 比较的计时质量失败[8]。

## 7.3 Apple、Hygon、AMD：不同结论等级

Apple Metal 已有各自设备上的 TaskLab 记录。M4 两个固定行归约任务相对其固定基线分别保留 1.107 和 1.138 倍的历史晋升观察；M1 Pro 的固定小形状 GEMM+SiLU 相对朴素基线曾确认 23.604 倍。大比值反映该具体基线与形状，不能被写成对专家库或全部 Apple family 的一般优势。M2 的 GEMM 搜索有描述性读数，但未通过测量稳定性门。Metal 计时包含整个 command buffer 并按重复 dispatch 均摊，不与 CUDA 的 CUPTI 微秒值横向比较[9]。

Hygon BW1101 的保留 sweep 同时出现确认加速、确认变慢和分辨能力待查。例如逐通道矩任务在原记录中为 9.439/6.719  $\mu$ s (1.405 倍)，prelu 为 5.439/9.759  $\mu$ s (确认变慢)；多项短内核两臂恰好报告 5.439  $\mu$ s，来源将它们标为测量分辨能力待查，而不据此断言性能相同。汇总中部分历史形状未转录，报告正文引用案例时要保留原 Campaign 定位，不能补造形状[10]。AMD gfx1151 已有 Triton/HSACO 路线及设备调查，但两种计时器的绝对时间尚未对齐；当前发布行没有合格加速比，也没有完整 Agent 优化结论[11]。

## 7.4 MetaX：单内核、完整输出和实际 Run 分别验收

C550 的本地证据应按三种不同范围阅读：(1) 固定单内核 M17 GEMM 把 M tile 64 改为 32，独立确认 72.448/58.368  $\mu$ s，得到 1.241 倍本机局部收益；(2) GQA、MLA、MoE 等 17 个变体共 86/86 个完整输出 case 数值通过，相应收据没有计时样本；(3) 当前集成文档记录了十轮 C550 Agent Run

**表 6** 三个平台的代表性观察按各自基线和质量状态呈现。数值属于原报告的历史 Workload 和软件，不能据此求跨平台平均。

| 目标           | 固定案例           | 观察              | 可陈述的范围           |
|--------------|----------------|-----------------|------------------|
| Apple M4     | 两项 FP32 行归约    | 1.107 / 1.138 倍 | 各自任务的历史晋升记录      |
| Apple M1 Pro | GEMM+SiLU, 小形状 | 23.604 倍        | 相对朴素任务基线, 非专家库对照 |
| Hygon BW1101 | 逐通道矩           | 1.405 倍         | 首个保留合格 Run 的本机结果 |
| Hygon BW1101 | prelu          | 0.557 倍         | 确认变慢, 应保留负结果     |
| AMD gfx1151  | RMSNorm smoke  | 延迟缺失            | 路线调查; 计时器未对齐     |

的反馈与确认路径, 12.032/11.008  $\mu\text{s}$  的配对结果只属于那次固定基线和 `local_serialized` 容器范围。三种事实不能拼成“完整 Program 的 Agent 自动优化已获得 1.241 倍”的陈述。MACA 路线的身份、MCPTI 样本、原生 profiler 与多阶段 attribution 的限制见 [C550 专题](#)和 [当前集成案例](#)。

## 7.5 案例 A: C550 的局部 tiling 收益

**1.241 倍** C550 • FP16 GEMM •  $M = 17, N = 128, K = 2048$

基线 72.448  $\mu\text{s}$   $\rightarrow$  候选 58.368  $\mu\text{s}$ ; M tile 64  $\rightarrow$  32。独立确认的正确性与测量质量门通过。

这是 `local_serialized` 范围内的本机 authoring 对照, 支持这个形状上的局部调度改进。它没有比较“向 Agent 提供 NVIDIA 经验”与“不提供经验”的处理差异, 也不是完整 Program 的自动优化 Run[12]。同一专题中的 17 个 Workload 变体、86/86 个完整输出 case 属于正确性证据; 其计时为缺失, 不能拿来扩大上述收益范围。

## 7.6 案例 B: B300 的对齐条件与质量门

**1.081 倍** B300 • FlashInfer 任务 026 • 守卫式对齐

固定 generic sliced-w8 对照 6.4005  $\mu\text{s}$   $\rightarrow$  候选 5.920  $\mu\text{s}$ ; 同次候选对照边通过计时质量门。

同一候选对外部参考的边没有通过 CV 质量门, 因此不能从该案例推出对外部参考的合格胜负。另一任务 008 的完整两阶段 Program 已通过 2550 份数值快照, 但新候选相对对照及外部参考的计时边均未通过质量门: 它证明完整程序可运行和算对, 不提供合格的性能结论 [8]。

## 7.7 两个局部案例的结论范围

两个局部改进来自不同硬件、Workload、源码与计时机制, 不能平均, 也不能作为一个跨硬件机制已迁移的轨迹。负结果、`close_null`、未完成和质量失败与正结果同样保留。截至此报告所读的 [当前状态页](#), 项目没有统一的已验收 Study Report 索引; 本文不计算全项目成功率或总加速比。

## 7.8 历史 B200 案例的放置方式

2026 年 9 月旧版报告 给出两个值得保留的有界案例。QSA 在固定  $T = 32768$  的完整 Program 上，历史确认候选与同次 CUDA 黑盒参考约为 127.8/676.4 ms；它说明程序表示和评测能承载多阶段候选，不能归因于一次同预算的智能体处理比较。Flash-KMeans 在历史 B200 固定形状上每组有三次独立搜索，Open-Cake 与直接 CUDA 作者环境的确认终点中位数约为 1.512/5.342 ms，历史比值约 3.53；该实验使用 150k token、max 努力与自己的历史源码和监管合同，不能和 CAKE 论文的 80M、xhigh 条件直接相除。正文保留这两项作为历史实验快照，附录列完整 Workload、软件身份、样本口径及来源；在当前提交上没有为它们重新赋予设备或监管资格 [6]。

## 8 与 CAKE 的对照：方法、重叠任务与差距

比较首先要确定对象：CAKE 论文的结论、公开的 CAKE 生成产物、本仓库的独立实现，以及某个固定 Workload 上的外部参考，是四种不同来源。表7 将研究问题放到同一行，再标出能够直接对应和不能直接对应的部分。原论文在 B200 Flash-KMeans clean-start 上报告 Cake IR 与直接 CUDA/PTX 的三次运行及 80M token 终点；本仓库历史 B200 实验的作者环境、努力级别与预算不同。两个实验可以比较方法条件和待解问题，不能把两组数值拼成“复现论文收益”[1, 6]。

**表 7** CAKE 论文与本仓库的对应范围。paper\_reported 与本地观察来源分开；一格中的“可实现”不代表已经形成匹配的性能实验。

| 问题       | CAKE 论文 v1                                            | 本仓库当前可读证据                                              |
|----------|-------------------------------------------------------|--------------------------------------------------------|
| 执行计划     | 硬件显式 Cake IR；NVIDIA Ampere-Blackwell                  | 独立 typed Schedule/Program；五 Vendor、八精确 Target 文档       |
| 静态反馈     | Verifier 四类门与目标校准成本排序                                 | 共享四类诊断和后端预检；无目标校准时报告覆盖限制                               |
| 作者处理     | B200、固定模型和 scaffold，Cake 对直接 CUDA/PTX，3+3 次，80M token | 历史 B200 配对运行有自己的 150k/max 条件；当前无可与论文对齐的 clean-start 估计 |
| 专家端点     | KDA、TinyGEMM、Alpha-MoE 等生产任务及服务/portfolio 结果          | 部分固定任务复现与完整输出；大范围 dispatcher、服务及部分原语仍缺                 |
| 非 NVIDIA | 原论文未测目标侧 lowering 与成本                                 | Apple、AMD、Hygon、MetaX 逐目标实现；正确性和计时资格不均                 |

CAKE 论文表 2 在 B200 Flash-KMeans 的相同 80M-token 截点，报告 Cake IR 三次运行的最优候选相对固定 FlashML 基线的中位数为 1.144 倍，直接 CUDA/PTX 为 0.928 倍；论文给出的参考基线是 0.938 ms。这些是 paper\_reported 数字，不是本仓库的执行记录。本仓库历史 B200 配对搜索的 150k-token 结果使用不同努力级别、作者环境与基线，故下文不画跨这两项研究的相对性能柱状图 [1, 6]。

### 8.1 TinyGEMM2：同形状的诚实差距

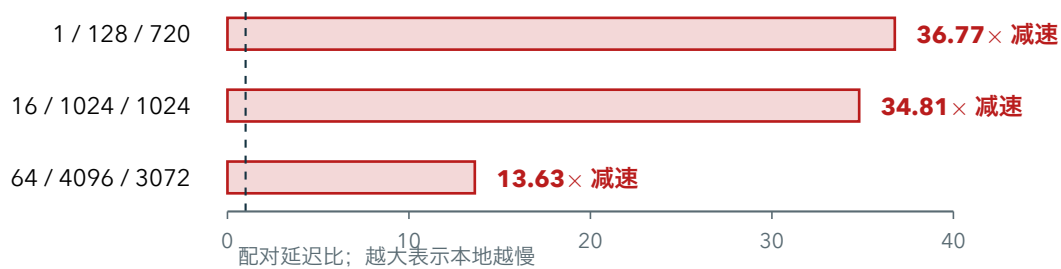
与完全不同任务的相对加速比相比，B300 的 TinyGEMM2 有更直接的任务重叠。本仓库固定三个 BF16+bias 形状，使用独立 FP64 数学 oracle、官方导出产物作严格 bitwise BF16 peer gate：本地候选 30/30 检查和官方导出 15/15 检查通过。五轮交替顺序的 CUPTI 冷 L2 测量按成对收据保留；表8显示本地 stage4 候选与官方导出 stage4 在这些形状上的差距。它证明数值重写可以构造，也**明确暴**

露当前 TMA、角色流水线和更大形状覆盖尚未追上对应生成产物；后者的原始 35/239 形状与服务验证不能从三行推断 [13]。

**表 8** B300 TinyGEMM2 固定形状的历史原报告中位数 ( $\mu s$ )。减速倍数来自原始成对比值，不以表中四舍五入数重新计算；这些行不是 CAKE 论文的 clean-start 对照。

| $B/N/K$      | CAKE 导出 stage4 | 本地 stage4 | 本地相对减速  |
|--------------|----------------|-----------|---------|
| 1/128/720    | 2.720          | 100.192   | 36.77 倍 |
| 16/1024/1024 | 3.040          | 105.825   | 34.81 倍 |
| 64/4096/3072 | 21.216         | 289.186   | 13.63 倍 |

本地 stage4 相对 CAKE 导出 stage4 的描述性减速



**图 6** B300 上同一 TinyGEMM2 Workload、同一形状与严格 bitwise BF16 门下，本地 stage4 相对官方 CAKE 导出 stage4 的历史配对中位数减速。只展示三项已执行形状，不显示跨轮不确定性；原始样本与质量观察见专题。本图不是 CAKE 论文的 80M-token 智能体实验。

原报告没有测得可以把这段减速唯一归于寄存器 spill、占用率或访存量的 profiler 归因，因此这些只能作为下一步假设。KDA prefill/decode 与 Alpha-MoE 的局部探针和规格也不等于完整候选或性能结果。对照章节保留能完成的任务范围、明确缺口与后续验收，避免把一个 probe 当作一整个家族的复现 [13]。

## 9 跨硬件知识迁移：机制与待验证估计量

### 9.1 可复用的单位与负责位置

候选优化可能包含两种不同的知识：一个带适用条件的语义改写，以及在某个硬件上选出的 tile、执行组或 pipeline 参数。前者可在 IR 中形成显式 pass，声明前提并返回完整候选或拒绝原因；后者要在目标 Target 上重新选择和验证。以私有中间张量的融合为例，数据流私有性、舍入、消费者和生命周期是机制条件；寄存器占用与具体收益属于目标设备事实。来源设备的速度比不随机制迁移 [14]。当一次研究没有可推广改写时，记录 No promotion 是正常结果。

### 9.2 E/P 四组实验

仓库已实现对经验材料  $E$  与显式变换调用权  $P$  的四格隔离和软件协议，但尚没有共同目标、基线与预算下的实机迁移收益实验。四组分别为  $E0P0$ 、 $E1P0$ 、 $E0P1$  和  $E1P1$ ； $P1$  的最小接口合同本身会暴露必要机制信息，因而  $E$  的效果只指额外说明与案例。各组共享 typed IR、Target、oracle、计时质量门和可表达的基础能力。材料与调用权限须通过真实隔离执行，不能只由提示词声称关闭 [15]。



图 7  $E$  只控制额外说明与案例;  $P$  控制显式 pass 调用权。四格是预注册处理设计, 图中没有实机处理效果;  $P1$  的最小接口本身会透露必要机制信息。

令  $p_{ep}$  为在预注册预算内经新鲜确认、达到本机预定改善阈值的 Run 比例。预定主要对比为

$$\begin{aligned}\Delta_E &= \frac{1}{2}[(p_{10} - p_{00}) + (p_{11} - p_{01})], \\ \Delta_P &= \frac{1}{2}[(p_{01} - p_{00}) + (p_{11} - p_{10})], \\ \Delta_{EP} &= p_{11} - p_{10} - p_{01} + p_{00}.\end{aligned}$$

独立 Run 是重复单位; 正式多任务研究以任务等权汇总并保留组内配对结构。搜索成本、失败、预算耗尽和基础设施缺失须按预注册规则保留。协议的存在证明问题可被提出和执行, 只有冻结处理后的设备实验才能估计上述三个量。

### 9.3 已到达工具链的离线迁移实例

当前迁移专题新增了一条 B300 到 Hygon BW1101 的有来源路径。源任务 pairwise\_sqdist 在 B300 固定 FP32 形状上, 候选把 K 维按 256 分块, 来源 Run 的五类输入前后检查和 CUPTI 配对质量通过, 相对本机固定基线  $411.7945/53.856 \mu s$  (7.646 倍)。移向 gfx938 时, Workload 的数学、张量、五类输入、oracle 与容差保持一致; 历史 Schedule v1 不能直接改目标, 先显式适配到 v2 execution\_groups。适配后, 固定 Compiler 接受并 lower 候选与同卡 starter, Hygon Triton 3.6.0 将两者编成 HSACO, 并通过 CPU 配对准入。

这条路径已经证明带有一次明确 IR 版本适配的机制能够到达目标工具链, 但尚未在 Hygon 加载、对完整输出判对或计时。B300 的 7.646 倍没有转移为 Hygon 的性能成绩, 也不是  $E/P$  四格对照的因果结果。目标设备阶段需要五类 case、同卡基线配对、声明计时边界和独立确认, 再决定是否把这一机制记作有效迁移。

## 10 限制、可复查性与结论

本文的最强软件结论是当前源码上的表示、检查、生成和控制协议, 以及 179 项语料预期相符。最强设备结论是指定源码、目标与固定 Workload 上的正确性和局部合格改善。当前材料不足以估计跨硬件经验迁移的因果收益、普遍性能优势, 或模型与服务层面的端到端收益。静态分析的覆盖也受目标声明和后端实际资源分配限制; 缺乏目标计时校准时应报告未知, 而不是给出没有根据的延迟预测。

后续工作以同一职责边界推进: 补齐各平台完整 Program 与目标框架验收; 在有原始证据的目标上校准测量与成本估计; 将重复的失败形成有反例的 Verifier 规则、IR 能力或 guarded pass; 最后用预注册 E/P 实验检验迁移增量。这个顺序让 Kernel 结果、Compiler 演进与研究归因各有自己

的证据和版本。

### 10.1 解释结果时的四项限制

**实现来源。**CAKE 论文公开了系统机制与聚合结果，但原始编译器版本、完整 IR/Verifier、agent scaffold、原始 clean-start 轨迹和部分计时细节并未作为可逐字节重建的同一包公开。本仓库的 Compiler 是独立实现，不能把相似接口或同名任务说成原实现的逐字节复现 [6]。

**平台不均。**五个 Vendor、八个 Target 文档说明目标集合与离线准入；每个目标的工具链、数值合同、计时器和 profiler 资格仍由自己的收据决定。apple\_gpu\_family7 和 apple\_gpu\_family9 在当前 179 项 Corpus Gate 中没有被检查；其他目标的通过数不能补上这种缺席。MetaX 的完整输出 case 与 AMD 的计时器调查也不能占用彼此的结论等级 [7, 2]。

**局部性能与应用。**单形状候选可能在自己的基线前改善，同时仍落后于外部专家实现。多形状 dispatcher 的守卫、fallback、尾部和框架开销还需要独立验证。本文的 QSA 历史案例只计完整 Program 的 GPU 区间，不能外推为模型或服务延迟；M17 的单内核提升也不自动成为多阶段 Program 提升。

**研究归因。**普通工程 Run、provider 资格、软件消融夹具及有条件的性能观察都能帮助完善系统，但只有预注册处理、独立 Run、共同确认和完整缺失规则才能估计作者环境或机制材料的因果效应。当前 E/P 四组尚无这种实机结果。

### 10.2 下一步的可检验目标

后续工作首先补齐目标侧完整输出、计时区间、状态重置与测量分辨能力，再扩展同一任务的 Program 和框架接入。在固定来源平台机制与目标开发集后，冻结 E/P 四组的材料、工具权限与任务清单；各目标只相对自己的本机基线报告确认率、搜索成本和条件性能。对实际反复出现的表达或分析缺口，保留触发 Finding、反例、后继提交及其验证，再观察新能力是否帮助后续任务。这样才能逐步回答“协同演进产生了什么可复用能力”，而不靠跨平台拼接的加速比。

## A 术语、源码入口与完整示例

表 9 文中的术语按负责事实区分。英文名是 API 查找键，表中中文名用于叙事。

| 对象       | 本报告中的固定含义                   |
|----------|-----------------------------|
| Workload | 外部算子语义、输入域、oracle 与数值容差     |
| Schedule | 单内核的操作、资源、执行组、访问与同步承诺       |
| Program  | 公共张量、多个叶子 Schedule 的阶段顺序和绑定 |
| Target   | 精确设备声明；事实未建模时不借相邻型号         |
| Run      | 一次冻结的作者环境、候选、预算与终态          |
| Study    | 运行前固定研究处理、分配、估计量和分析规则       |
| Finding  | 可定位、可归属的结构、目标或后端诊断          |

当前 [Softmax 文件](#) 是第 4 节运行示例的完整可读源码：

```
from open_cake_ir.compiler import frontend as cake

@cake.schedule(name="softmax-b8-smoke", target="sm_100a", backend="triton",
               entry_point="cake_softmax_b8-smoke",
               residency={"ctas_per_multiprocessor": 4,
                         "registers_per_thread": 96})
def softmax(lm, x: cake.Tensor((8, 512, 128), "fp32"),
            y: cake.Tensor((8, 512, 128), "fp32", mode="output")):
    compute = lm.role(execution_groups=[0, 1, 2, 3])
    row_block = lm.program(x, axis=0, dimension=1, tile=64)
    batch = lm.program(x, axis=1, dimension=0, tile=1)
    with compute:
        x_tile = lm.load(x[batch, row_block, :], reuse="reused", id="load_x")
        rowmax = lm.reduce(x_tile, op="max", axis=1, scope="cta", id="row_max")
        shifted = lm.sub(x_tile, lm.broadcast(rowmax, axis=0), id="shift")
        weights = lm.exp(shifted, id="exponentiate")
        rowsum = lm.reduce(weights, op="sum", axis=1, scope="cta", id="row_sum")
        y_tile = lm.div(weights, lm.broadcast(rowsum, axis=0), id="normalize")
        lm.store(y[batch, row_block, :], y_tile, id="store_y")
```

这里的编写接口以 Python AST 构造与 JSON 相同的规范计划；`Compiler.load()` 默认读取项目的当前配置。复制这个示例不会自动取得某台 GPU 的可运行产物或性能资格。现有 [Python 前端文档](#) 列出允许的语法及源码位置诊断。

## B 结果来源与历史身份

**表 10** 正文各类结果的第一阅读入口。报告页面是带范围的投影，原始运行收据继续拥有测量与监管事实。

| 范围               | 固定来源或索引                                                               |
|------------------|-----------------------------------------------------------------------|
| 当前 Compiler      | <a href="#">状态页</a> 、 <a href="#">版本清单</a> 与精确目标文档                    |
| B300 FlashInfer  | <a href="#">逐任务状态</a> ；每行链接自己的 Finding 与收据                            |
| B300 TinyGEMM2   | <a href="#">严格数值与成对计时报告</a>                                           |
| Apple、Hygon、AMD  | 各平台 <a href="#">发布汇总</a> 与原记录                                         |
| MetaX C550       | <a href="#">平台专题</a> 及 <a href="#">当前 Run 案例</a>                      |
| 迁移离线试点           | <a href="#">机制与证据边界</a>                                               |
| 历史               | <a href="#">2026-W36.1 旧报告及其证据附录</a> ；退役路径按 <code>history</code> 标签回放 |
| QSA/Flash-KMeans |                                                                       |

历史 Flash-KMeans 的六次终点属于固定  $B = 32, N = 65536, K = 1024, D = 128$ 、BF16 输入的 B200 Workload，三对延迟（Open-Cake/CUDA，ms）为 1.191845/3.898164、1.511927/7.130795 和 1.512163/5.342292；两组各取自己的中位数，范围不是置信区间。历史 QSA 的  $T = 32768$  完整 Program 候选和同次参考分别为 127.821446/676.423598 ms；该时间不含 Python 包装开销。这里只转录旧报告的有界结论与身份，不以新版 Compiler 重释旧原始记录。

## C 复现边界与报告构建

本文的数字来自已存在的报告、Finding 与原始收据定位；写作和 TeX 编译没有启动新的 provider、GPU Campaign 或计时。阅读者可从固定源码检出运行离线入口，看到 IR 构造、Assessment 与 lowering；设备结果则须在原任务的目标、工具链、参考访问和监管环境下重放。报告源文件位于 `docs/open-cake-ir-technical-report.tex`，在装有 XeLaTeX、中文字体和本仓库已有图像的环境中可直接编译。历史示意图保留其原始来源；其文字表达的能力范围按正文当前源码重新核对，不用旧图中的版本值替代当前状态。

## 阅读与引用说明

本文是作者维护的仓库技术报告，无 DOI、期刊或同行评审声明。引用设计时请固定本报告及其所读源码提交；引用性能时还需追溯案例自己的执行源码、精确目标、Workload、baseline、oracle、计时区间与质量门。以下文献中的仓库链接均固定到本文所读的 `ff02b8fe`，而实验原始收据的身份由各专题继续给出。正式英文引文题名沿用仓库 `CITATION.cff` 的 `preferred-citation`；封面的中文题名概括本文的研究对象。

## 参考文献

- [1] Z. Ye et al. *CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution*. arXiv:2608.12629v1, 2026. <https://arxiv.org/html/2608.12629v1>.
- [2] H. Qin. 系统架构与平台能力. open-cake-ir@ff02b8fe, 2026.
- [3] H. Qin. IR 指南. open-cake-ir@ff02b8fe, 2026.
- [4] H. Qin. Compiler 清单与 Target 文档示例. open-cake-ir@ff02b8fe, 2026.
- [5] H. Qin. 验收门. open-cake-ir@ff02b8fe, 2026.
- [6] H. Qin. 论文与本地证据合同. open-cake-ir@ff02b8fe, 2026.
- [7] H. Qin. 当前状态. open-cake-ir@ff02b8fe, 2026.
- [8] H. Qin. B300 FlashInfer 任务综述. open-cake-ir@ff02b8fe, 2026.
- [9] H. Qin. Apple 平台结果. open-cake-ir@ff02b8fe, 2026.
- [10] H. Qin. Hygon DCU 平台结果. open-cake-ir@ff02b8fe, 2026.
- [11] H. Qin. AMD 平台结果. open-cake-ir@ff02b8fe, 2026.
- [12] H. Qin. C550 专题与 M17 独立确认. open-cake-ir@ff02b8fe, 2026.
- [13] H. Qin. B300 CAKE 重叠任务对照. open-cake-ir@ff02b8fe, 2026.
- [14] H. Qin. 跨硬件优化知识迁移机制. open-cake-ir@ff02b8fe, 2026.
- [15] H. Qin. E/P 消融实验设计. open-cake-ir@ff02b8fe, 2026.